



AutoCT Automation (Python 🐍)

Adding Automated Review at 99% Accuracy

Trevor J. Allen

Problem Statement

The Trader team had **no automated path to decisioning**. Throughput was capped by headcount, volume was driven by upstream programs, and high-accuracy detection patterns sat unleveraged.



CAPACITY CONSTRAINT

(Headcount-Capped Throughput)

Every trader-flagged order required individual analyst review. Throughput was directly tied to FTE count, with no path to scale volume without adding headcount.



CROSS-PROGRAM IMPACT

(Upstream Spikes Land on D68)

When teams like AMP and ASO Fraud action at scale on FA2 and DGF blocks, the resulting queue cascades onto D68 trader blocks. Volume drivers sat outside the team's control.



TOOLING LEVERAGE

(Proven Queries, Manual Execution)

TFAs had already written hunting queries with 99%+ accuracy. Those insights existed, but had no operational path beyond an analyst manually running them. Proven patterns sat idle.

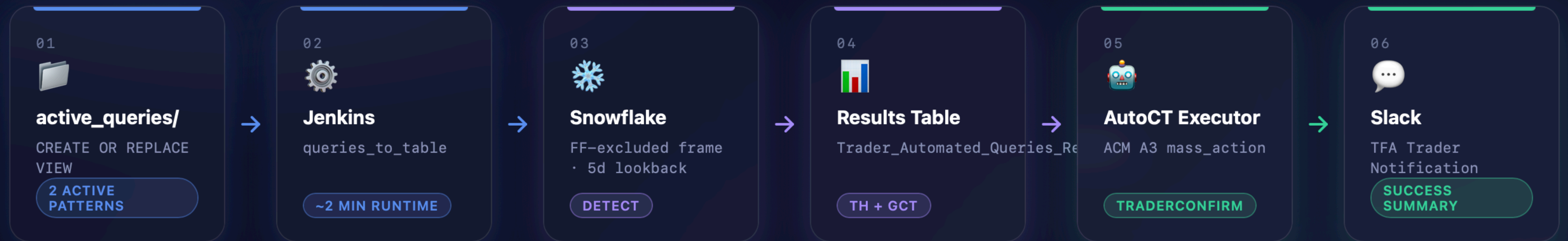
THE UNDERLYING OPPORTUNITY

Operationalize What Already Works

All three problems converge on the same gap. The team had proven detection patterns but no way to run them autonomously. Building tooling to consolidate analyst-developed queries into an automated decisioning pipeline does three things at once: it frees headcount from manual review, creates a defensive mechanism against upstream volume spikes, and unlocks the value of high-accuracy insights that had been sitting in analyst hands. The work was not in inventing new detection logic. It was in **extending an existing process with the tooling needed to scale it**.

AutoCT Pipeline

End-to-end Trader Hunting — automated review at >99% precision



DETECT

Pattern definitions → Jenkins

SQL patterns live in source control. Jenkins extracts the SELECT body and runs it against the latest frame on every job tick.

PROCESS

Snowflake → Results table

FF-excluded temp frame is enriched with the full 50+ column row set, deduped on order_identifier, and written to the results table.

ACTION

ACM TRADERCONFIRM → Slack

TH-only backlog is filtered against ASO ST-blocks, mass-actioned through ACM A3 in 500-order batches, and reported to Slack.

2

ACTIVE PATTERNS

>99%

PRECISION

~2 min

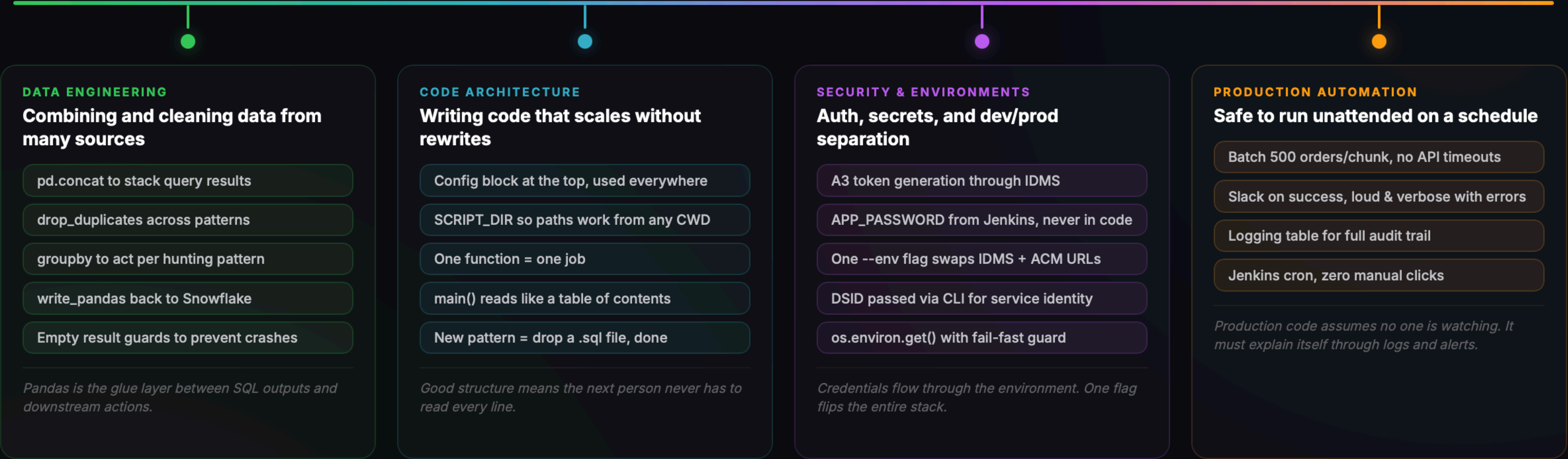
RUNTIME

0

ANALYST REVIEW

What I learned building this system.

Four skill branches that developed from course concepts into production habits across the Trader Hunting + AutoCT pipelines.



EX. OF CONCEPTS LEARNED: SCALABLE AUTH + ENVIRONMENT SWITCHING FROM TRADER_TFA_AUTOCT_EXECUTOR.PY

● ● ● trader_tfa_autoct_executor.py SECURITY & ENVIRONMENTS PATTERN

```
# All endpoints live in one dict – a single --env flag at runtime picks prod or dev, same code path either way
IDMS_ENDPOINTS = {"prod": "https://idmsservice.corp.apple.com/apptoapp/token/generate", "dev": "https://idmsservice-uat.corp.apple.com/apptoapp/token/generate"}
ACM_ENDPOINTS  = {"prod": "acm-nonregulated.g.apple.com", "dev": "acm-nonregulated-uat2.g.apple.com"}

# Credential is injected by Jenkins as a secret binding at job runtime – it never touches the repo
app_password = os.environ.get("APP_PASSWORD")
if not app_password: raise RuntimeError("APP_PASSWORD env var required") # fail loudly so silent auth failures never reach prod

# generate_a3_token posts APP_PASSWORD to IDMS, gets back a short-lived A3 token, then passes it to ACM
token = generate_a3_token(args.env, app_password) # one call, right environment, no branching logic needed downstream
```

Impact.

The trader team had no automated path to decisioning. The pipeline closes that gap and changes how the team operates day to day, from executing against known signals to investigating new ones.

ORDER VALUE AUTO ACTIONED

\$1.0B

across **854,059 orders** and **9 patterns**. Projected coverage of **228K orders** and **\$269M** every month.

8K to 10K

ORDERS PER DAY

\$8.98M

AVG VALUE PER DAY

99% +

ACCURACY OF PATTERNS RUNNING LIVE



TFA CAPACITY BOOST

A net new TFA in review capacity

8K to 10K orders decisioned daily with no analyst touch

\$8.98M in average daily order value covered

\$269M projected coverage every month

Capacity equivalent to one full TFA, no headcount added

The review queue is no longer the ceiling on how much we can action.



TIME SAVED

Hours back to investigate new trends

Known patterns drop into active_queries and run on their own

No more re executing the same review playbook daily

Analyst time redirects to emerging behavior and novel signals

Trend investigation replaces pattern execution

Time saved is only valuable if it goes somewhere. It goes to the work no automation can do.



WORKFLOW EFFICIENCY

A more scalable program takes shape

Dashboards surface trader rates by region and order type

Custom series built from high confidence signals

New patterns fold into the TFA workflow with no rebuild

Validated through backtesting before promotion

Each new pattern compounds the system. The workflow gets easier as it grows.

How TFAs Contribute

Any TFA can ship a new trader-hunting pattern. **Backtest it to 99%+ precision, open a PR, and the bot takes it from there.**



01



Draft a Snowflake SQL Pattern

Write your trader-hunting pattern as a **Snowflake SQL** file

.SQL

Iterate



02



Backtest via aors-innovation

Submit your backtesting SQL and/or backtested query through **aors-innovation**. **Precision must clear 99%** before promotion.

≥99% PRECISION



03



Submit a PR

Drop the `.sql` into `active_queries/` and open a PR against the main repo. Reviewer signs off.

PR · 1 FILE



04



Bot Takes Over

Auto Trader TFA Bot picks it up on the next run. **Frames, dedupes, actions via ACM, posts to Slack.** Zero ops work.

LIVE NEXT RUN



Your pattern, measurable impact. Every accepted pattern compounds the bot's coverage. More orders auto-confirmed, more analyst hours redirected from review files to net-new investigation. **Contributor credit is logged per TFA in the pattern results logging table.**

Next Stages

Where AutoCT goes from here — near-term migration work, long-term intelligence layer.

SHORT TERM

Target · July 31

3 Workstreams

Next 6 Weeks

- 1 Upload backtested patterns into the **active queries** folder
- 2 Sync infrastructure and data-generation functions with the **table generation logic** used by the broader TFA teams **by July 31**
- 3 Refactor **Slack notification** and **automated actioning logic** from the SDS-Fulfillment-TFA repo for reuse across **TFA spaces** **by July 31**

LONG TERM

Horizon · Q4 2026+

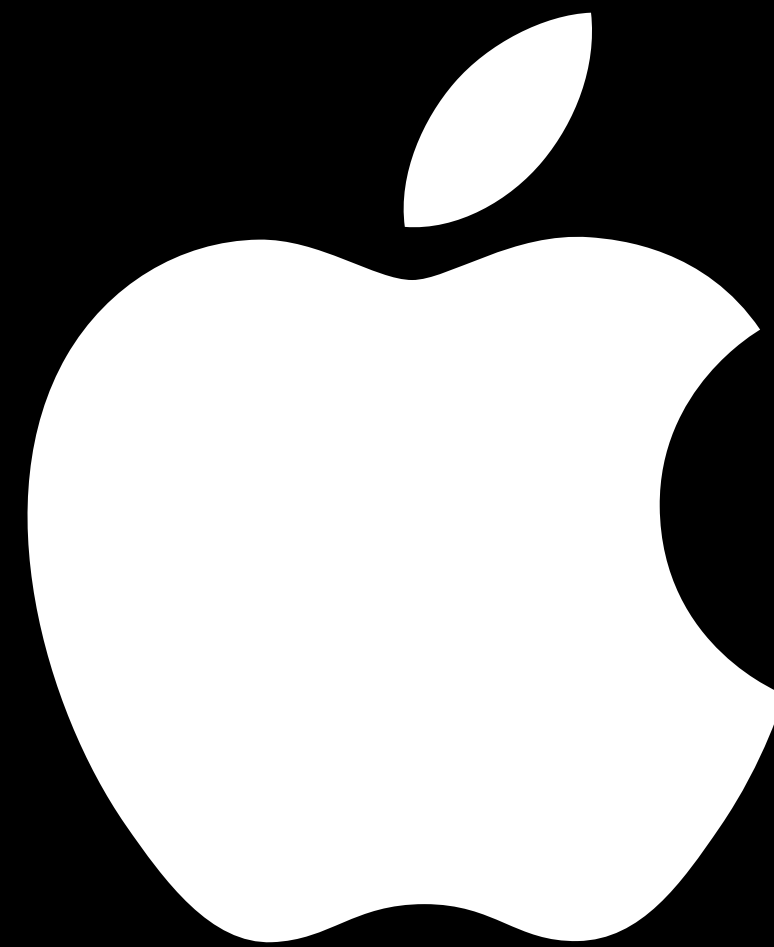
5 Workstreams

Next 6 Months & Onwards

- 1 Validate and add more patterns: **clustering, address counting, scoring models**, etc.
- 2 Improve job efficiency and build **visualizations** based on per-query performance
- 3 Incorporate **classification-model logic** into trader hunting patterns
- 4 Stand up an **active search / crawl agent** that combs the backlog for new trader activity using **vector and data-point similarity**
- 5 Onboard more **TFAs to contribute patterns** into the active queries folder

Questions for the team:

- **Any potential blockers you anticipate?**
- **Any questions about this new process?**
- All feedback is good feedback
- The process is still being perfected, your opinions and ideas matter!
- Observations and feedback can also be shared later



<https://github.pie.apple.com/ai/sds-fulfillment-tfa/tree/tjx-AutoCT>